

From Clock Edges to Safe Transfers: A deep dive into Clocking, CDC, and RDC

Chetan Sudarshan
Marvell Technology



Agenda

- **Timing basics**
- **Clocking**
- **CDC and Synchronization**
- **RDC and resolving RDC**



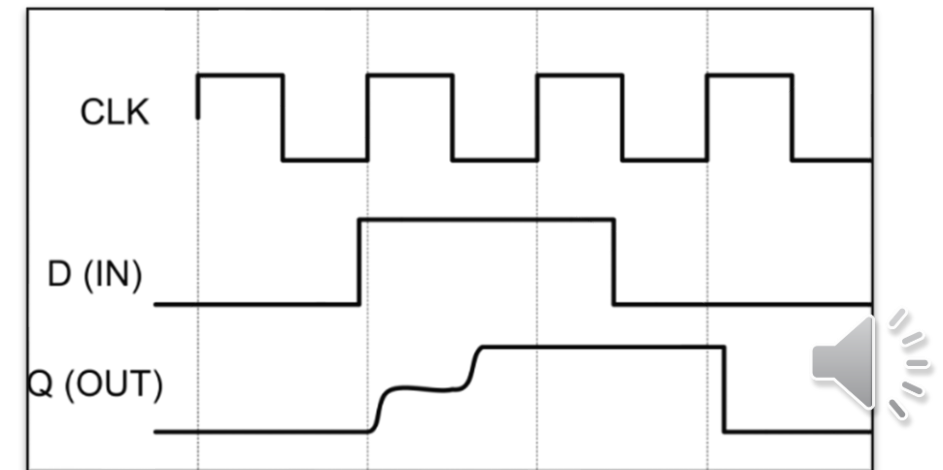
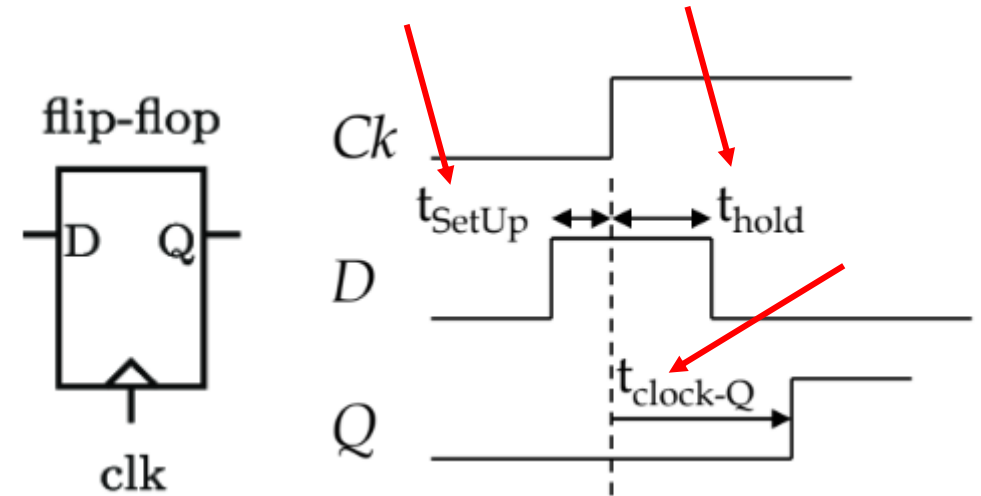
Agenda

- Introduction
- **Timing basics**
- Clocking
- CDC and Synchronization
- RDC and resolving RDC



Timing basics

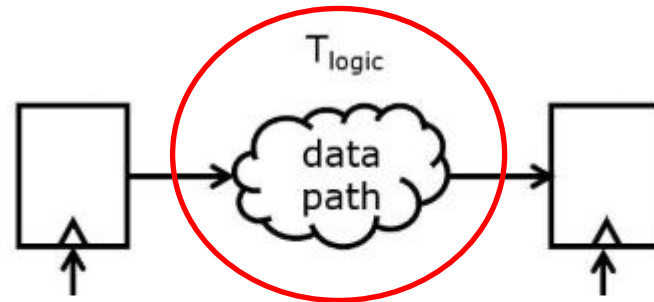
- **Static Timing Analysis (STA)**
 - Verify timing correctness of digital circuits without simulations
- **Setup time (t_{SetUp})**
 - Minimum amount of time before the clock edge, the data must be stable
- **Hold time (t_{hold})**
 - The minimum time after the clock edge, the data must remain stable
- **Clock-to-Q delay ($t_{\text{clock-Q}}$)**
 - Time from clock edge to output change of a flip-flop



Source: <https://colinoflynn.com/2020/12/experimenting-with-metastability-and-multiple-clocks-on-fpgas/>

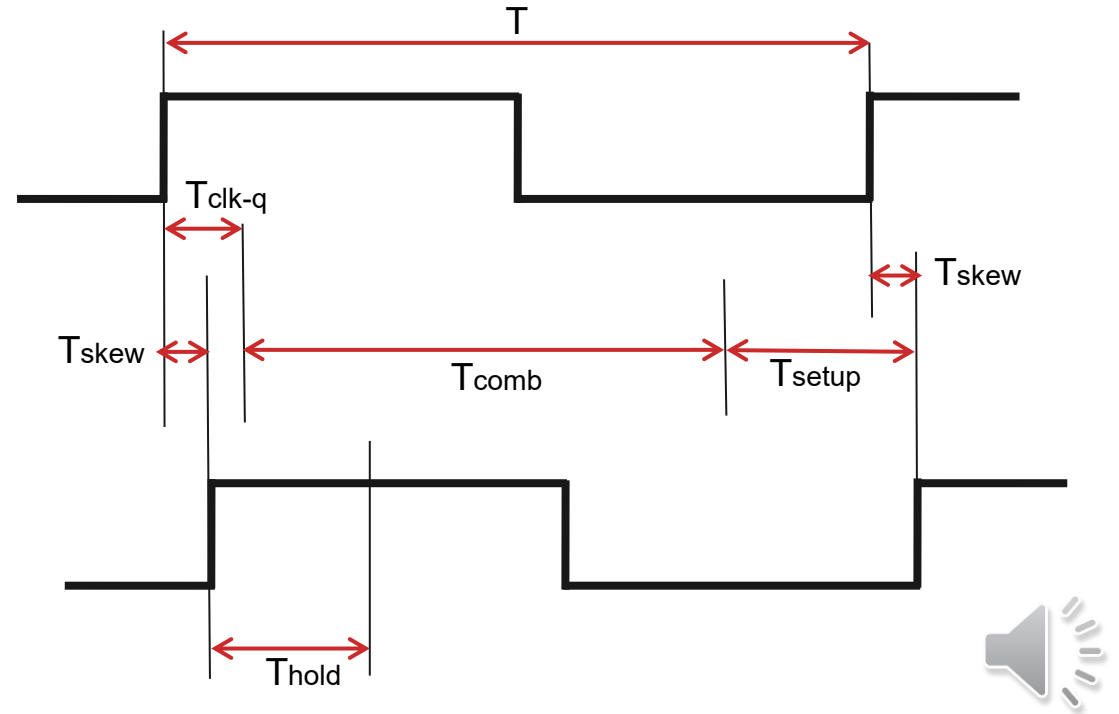
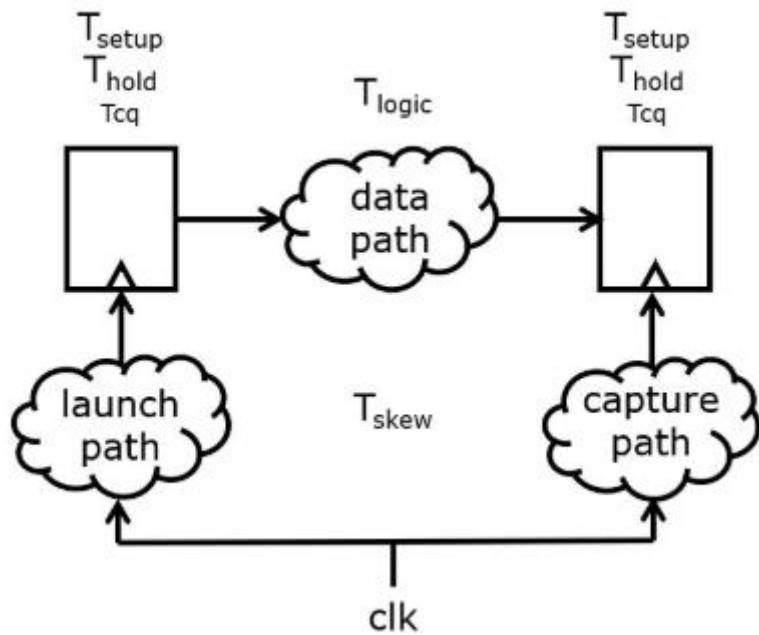
Timing basics

- **Combinational delay**
 - Delay between flip-flops caused by logic gates, muxes, wires



Timing basics

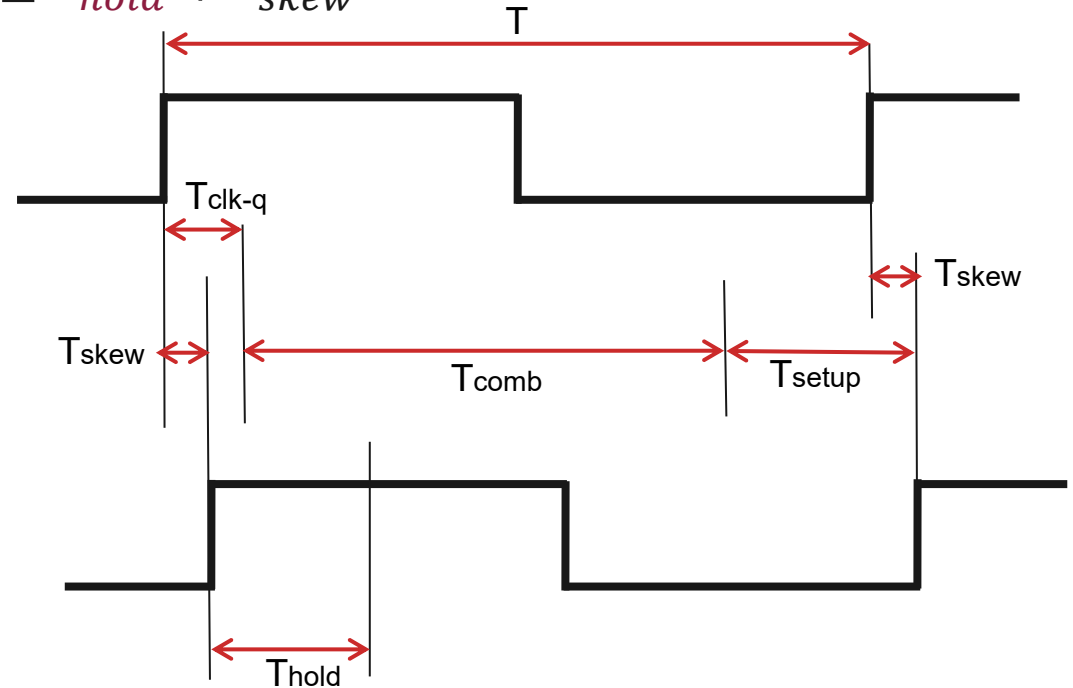
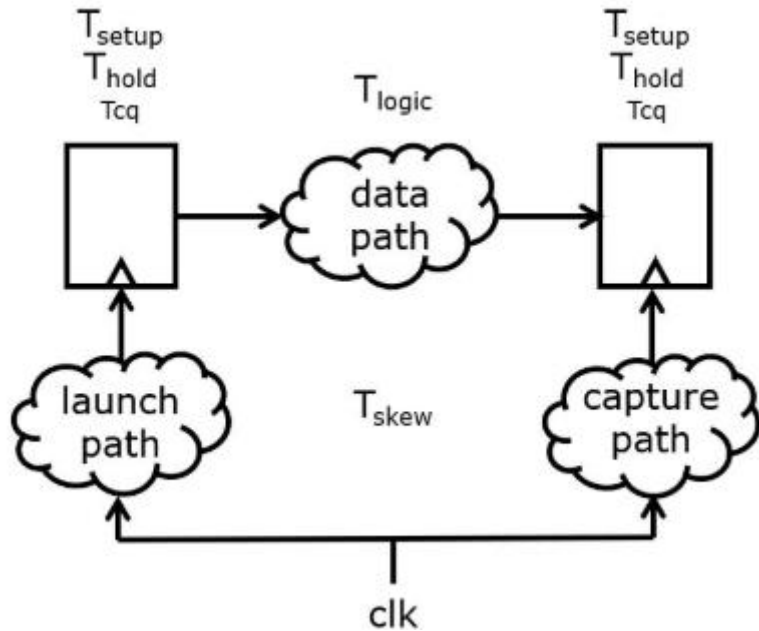
$$\boxed{T + T_{skew}} \geq T_{clk-q} + T_{comb} + T_{setup}$$
$$T_{clk-q} + T_{comb} \geq T_{hold} + T_{skew}$$



Timing basics

$$T + T_{skew} \geq T_{clk-Q} + T_{comb} + T_{setup}$$

$$T_{clk-Q} + T_{comb} \geq T_{hold} + T_{skew}$$



$$T_{slack} = (T + T_{skew}) - (T_{clk-Q} + T_{comb} + T_{setup})$$

$T_{slack} > 0$: timing is met

$T_{slack} < 0$: setup violation



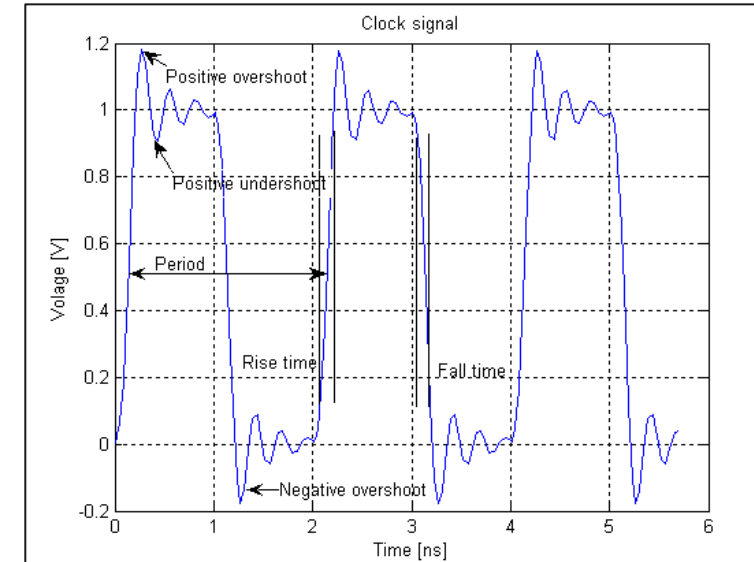
Agenda

- Introduction
- Timing basics
- **Clocking**
- CDC and Synchronization
- RDC and resolving RDC

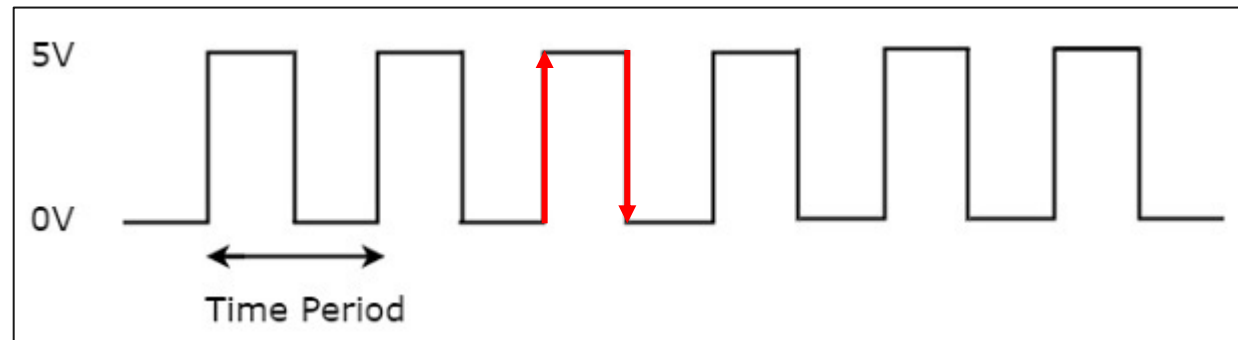


Clocking

- **What is a clock?**
 - A clock is a periodic signal oscillating between a high and a low state at a constant frequency.
 - Mainly used for synchronization and coordination of operations and data movement.

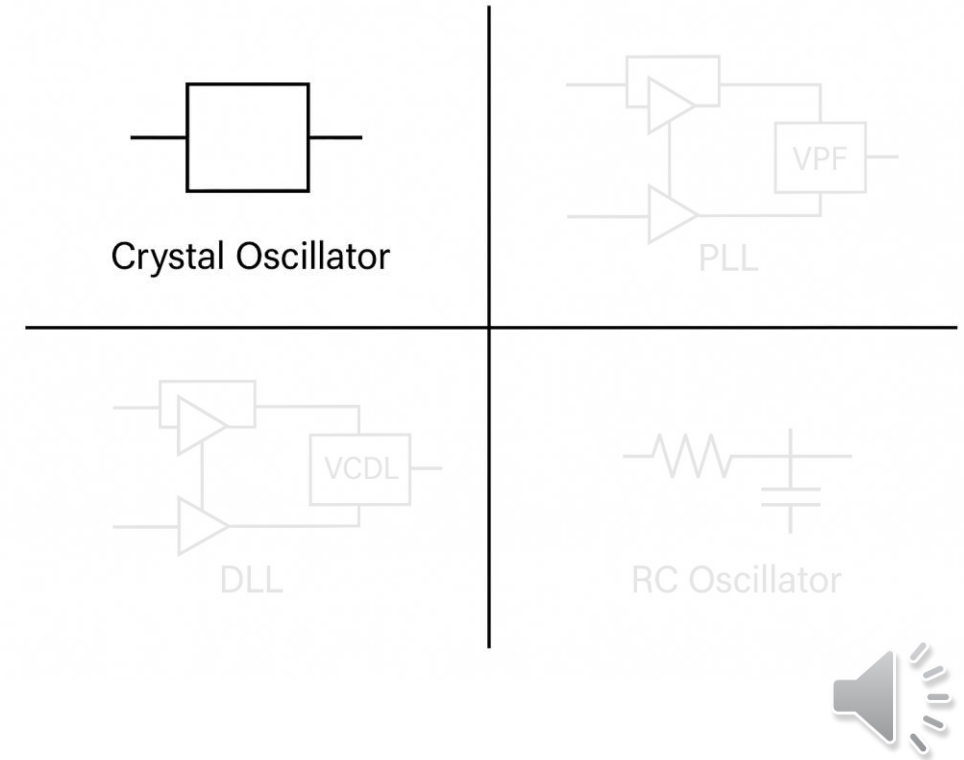


Source: Wikipedia



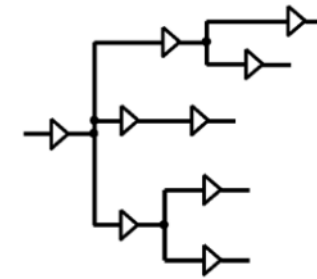
Types of clock sources

- Crystal Oscillator
 - Generates highly stable and accurate frequency using a quartz crystal
 - Used as a reference clock for PLLs
- Phase-Locked Loop (PLL)
 - Locks an internal oscillator to an external reference clock using a feedback control loop. Ability to multiply/divide frequency.
- Delay-Locked Loop (DLL)
 - Aligns the phase of the delayed version of the input clock using delay elements. (DDR)
- RC Oscillator
 - Uses resistors and capacitors to generate oscillations.



Clock distribution

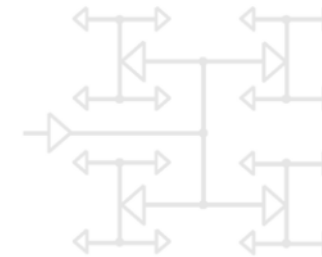
- Clock Tree
 - Hierarchical branching, distributing the clock
 - Buffer insertion used to balance delay, introduces clock skew
- X-Tree
 - Shaped like X, fanout is diagonally from the center.
 - Common in asymmetric layout or when load balance is complex, used in custom analog-mixed signal chips.
- H-Tree
 - Symmetrical layout to minimize skew
 - Used in mesh or grid-based designs like CPUs
- Tapered H-Tree
 - H-Tree but with different wire widths and buffer size
 - Helps handle high capacitance near the source.



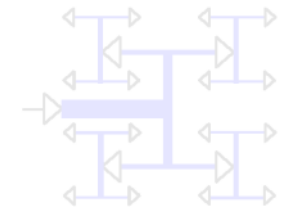
Tree



X-Tree



H-Tree

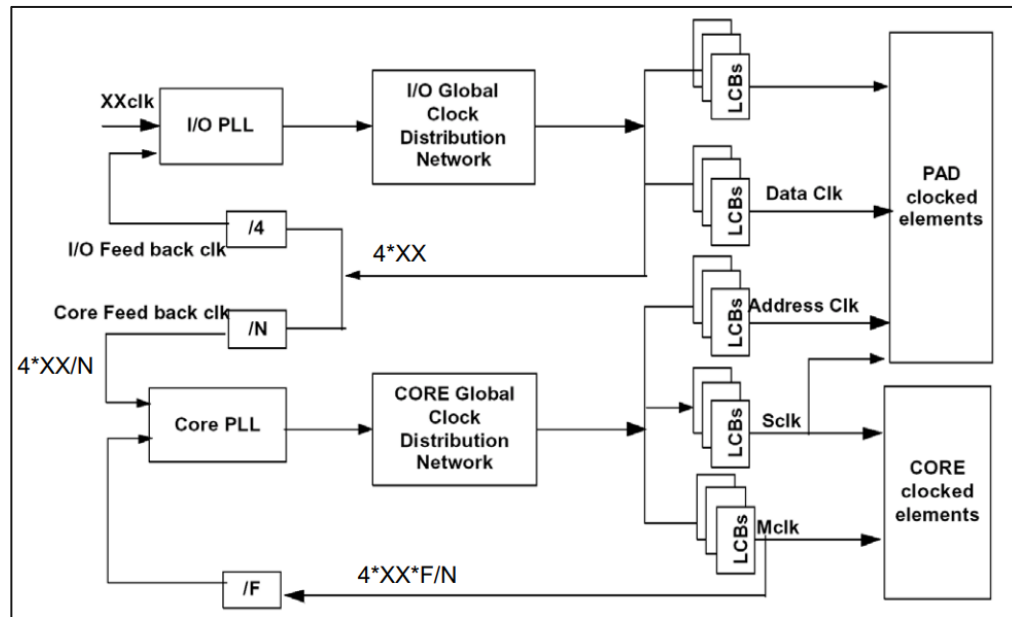


•Tapered H-Tree

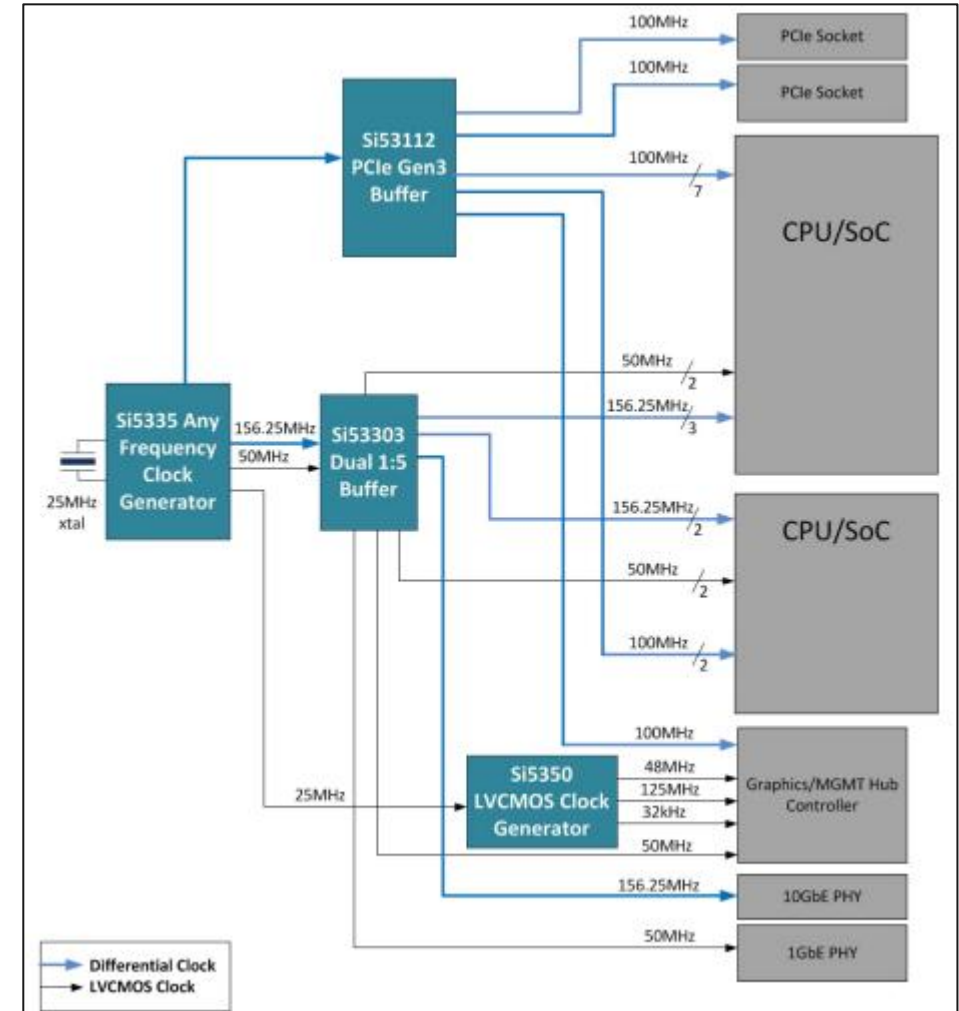
Source: https://users.ece.utexas.edu/~mcdermot/vlsi1/main/lectures/lecture_9.pdf



Clock distribution contd...



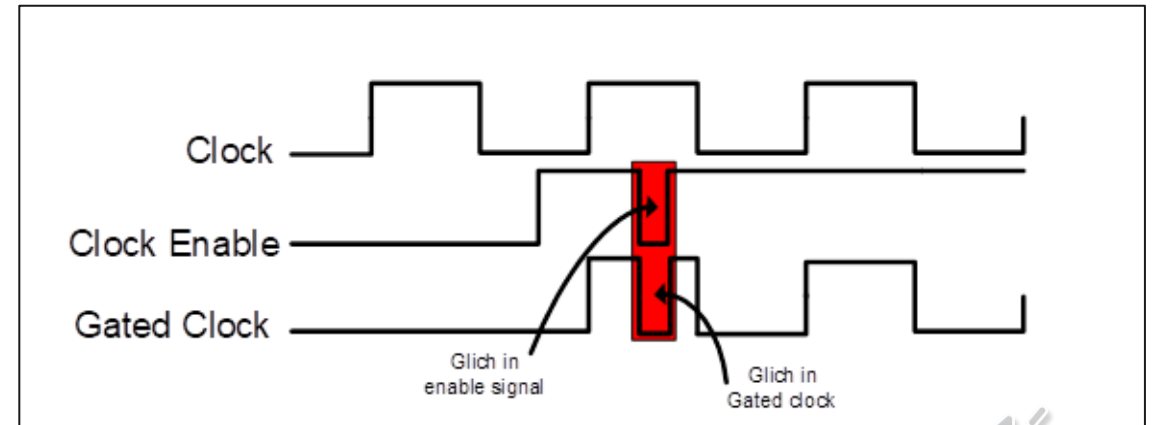
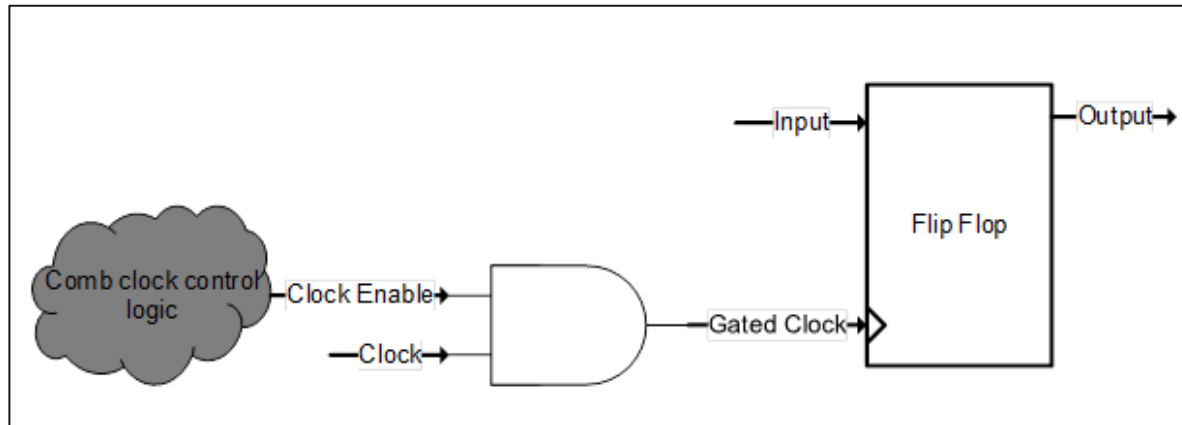
Source: https://users.ece.utexas.edu/~mcdermot/vlsi1/main/lectures/lecture_9.pdf



Source: <https://www.mouser.com/pdfdocs/clock-tree-101-timing-basics.pdf>

Clock Gating and Optimizations

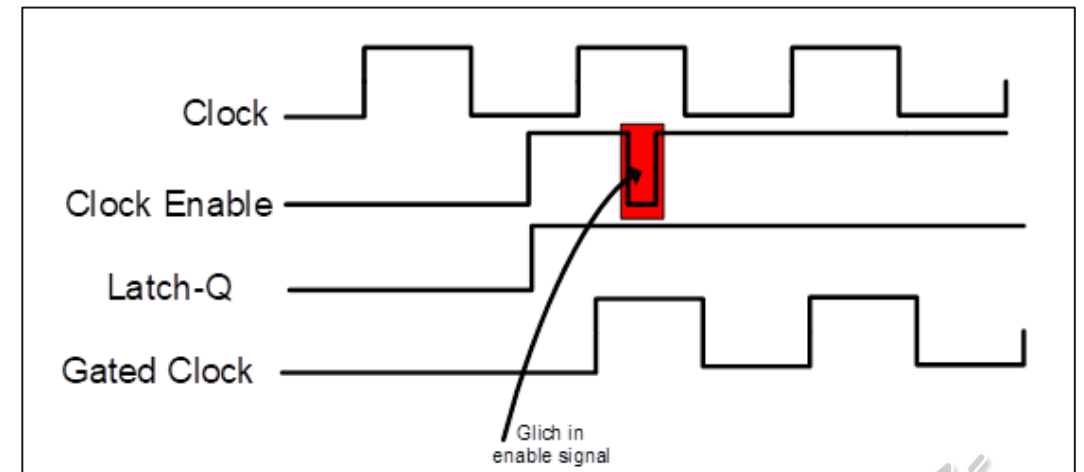
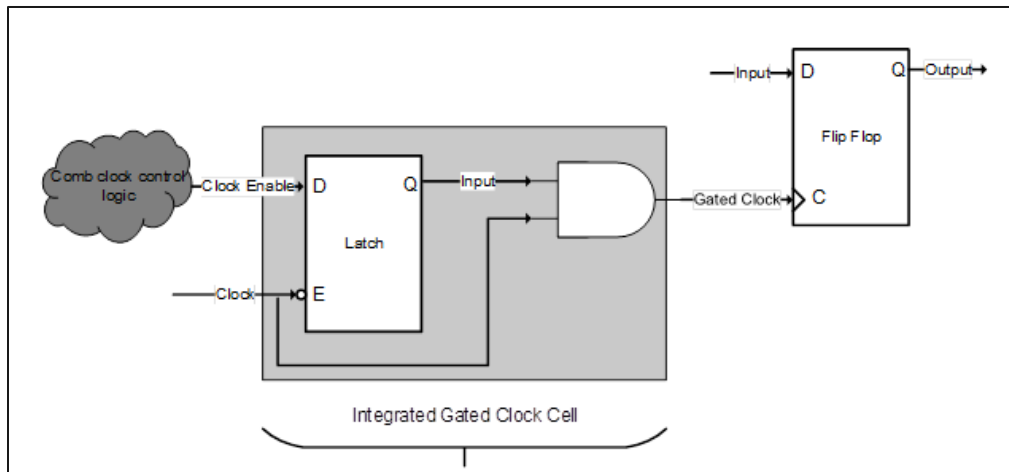
- Clock toggling consumes significant power even if logic is idle.
- Clock gating helps reduce dynamic power effectively
- Simple clock gating (clk & enable) is prone to glitching



Source: <https://anysilicon.com/the-ultimate-guide-to-clock-gating/>

Clock Gating and Optimizations

- Use an integrated gated clock cell to help with glitches
- Negative level-sensitive latch helps make sure we can avoid glitches



Source: <https://anysilicon.com/the-ultimate-guide-to-clock-gating/>

Agenda

- Introduction
- Timing basics
- Clocking
- **CDC and Synchronization**
- RDC and resolving RDC

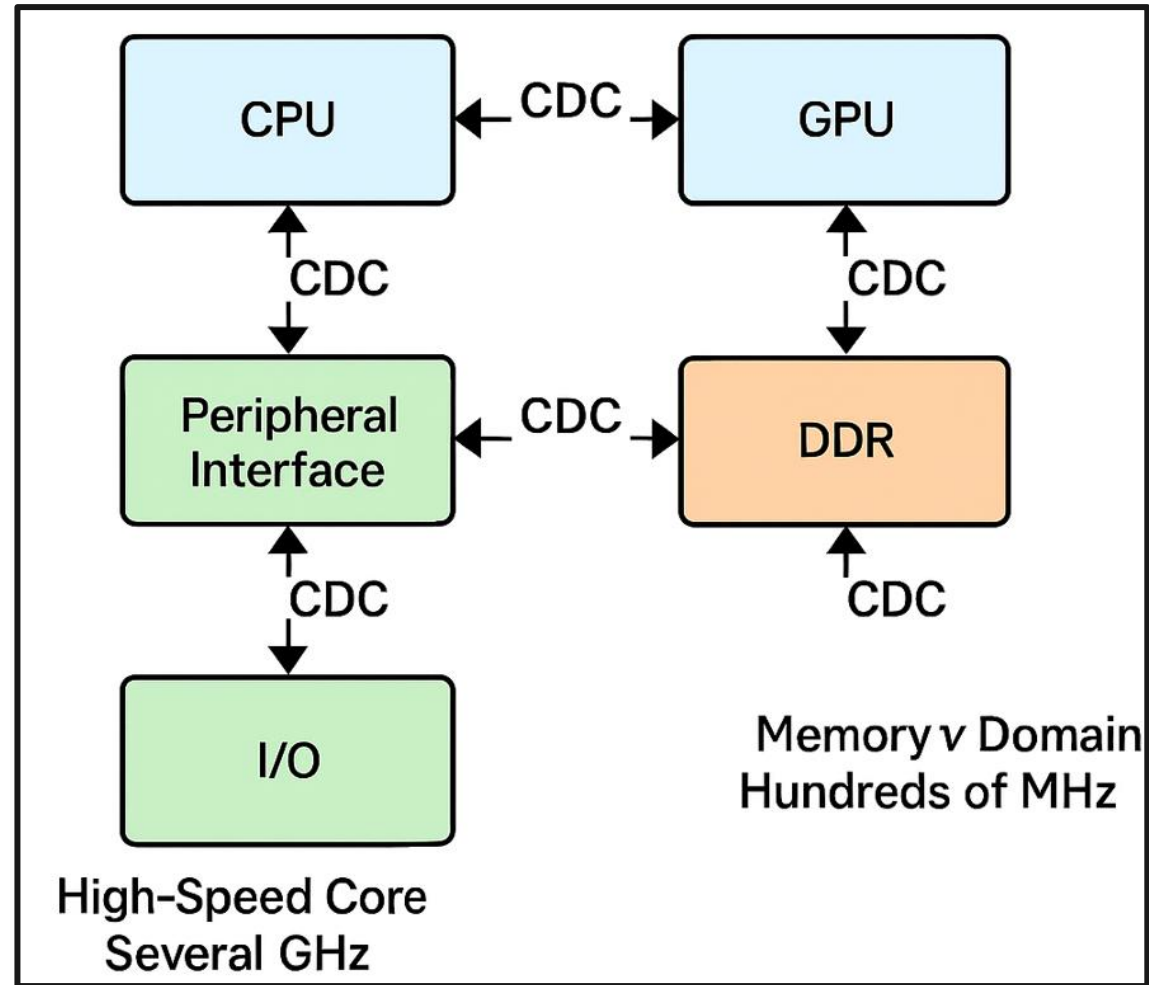


Types of clocks

- **Synchronous clocks**
 - Same source
 - Have an easily established timing relationship
 - Static Timing Analysis works
- **Asynchronous clocks**
 - From different sources
 - Timing relationship unknown or difficult to establish
 - Static Timing Analysis doesn't work

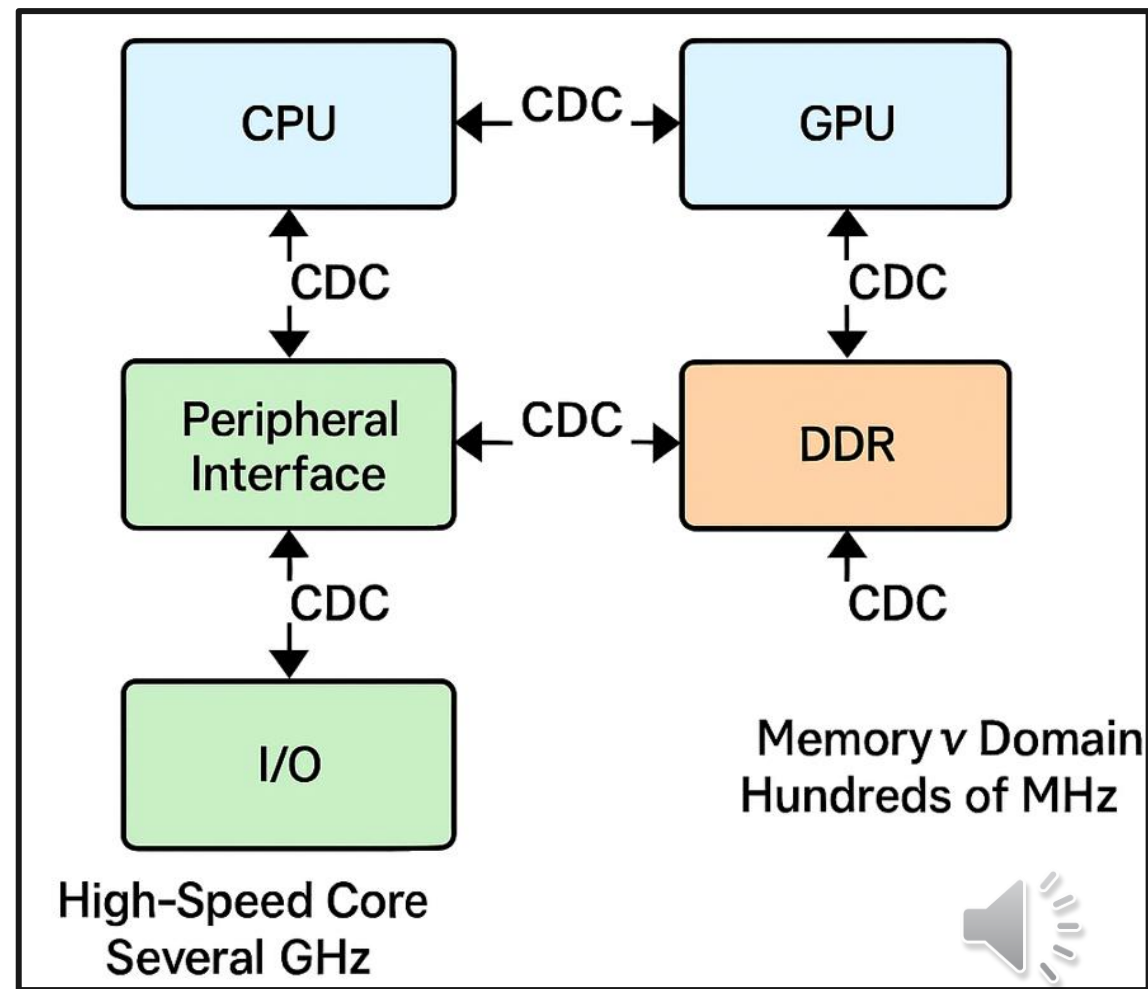


Why multi-domain?



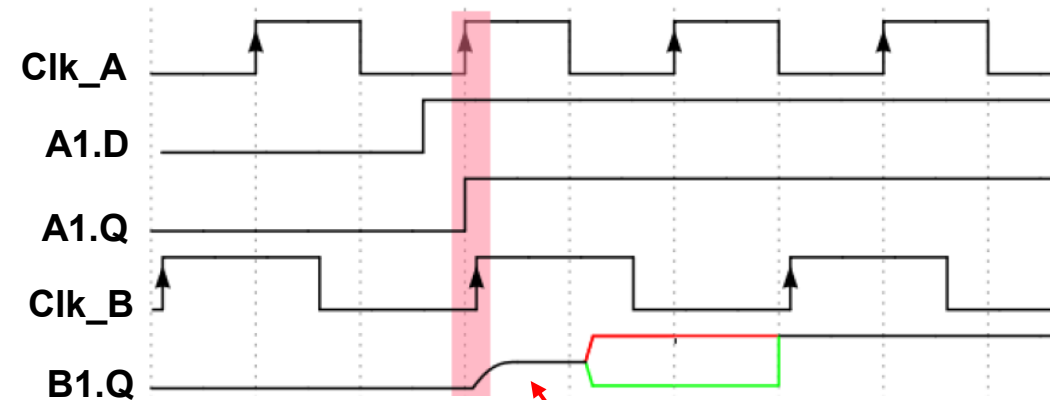
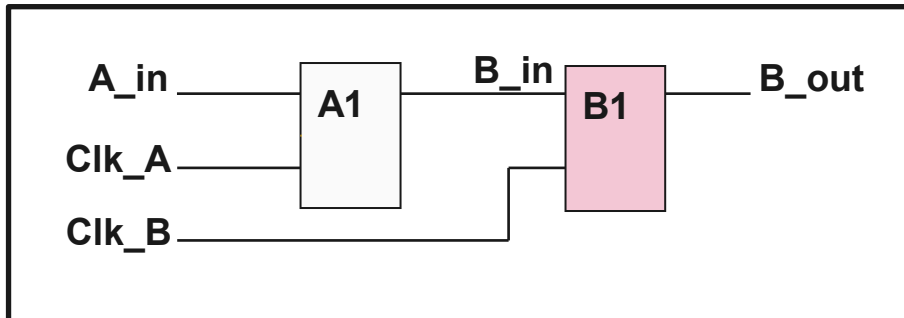
Why multi-domain?

- **Performance Optimization**
 - Different functional blocks (GPU, CPU, DSP, I/O, etc) require different operating speeds.
 - High-speed cores run at GHz range, while interface signals need tens of MHz.
- **Power efficiency**
 - Helps with enabling dynamic frequency scaling (DFS)
 - Lower frequency domain consumes less dynamic power
- **Scalability and Reusability**
 - Many 3rd party IPs have their own clocking requirements, such as DDR, PCIe, USB, etc.
 - Having clock domains modularized helps in easier development and verification
- **Clock source**
 - Not all clocks can be derived from a single PLL
 - There may be cases when we need async clocks from external sources



Synchronization

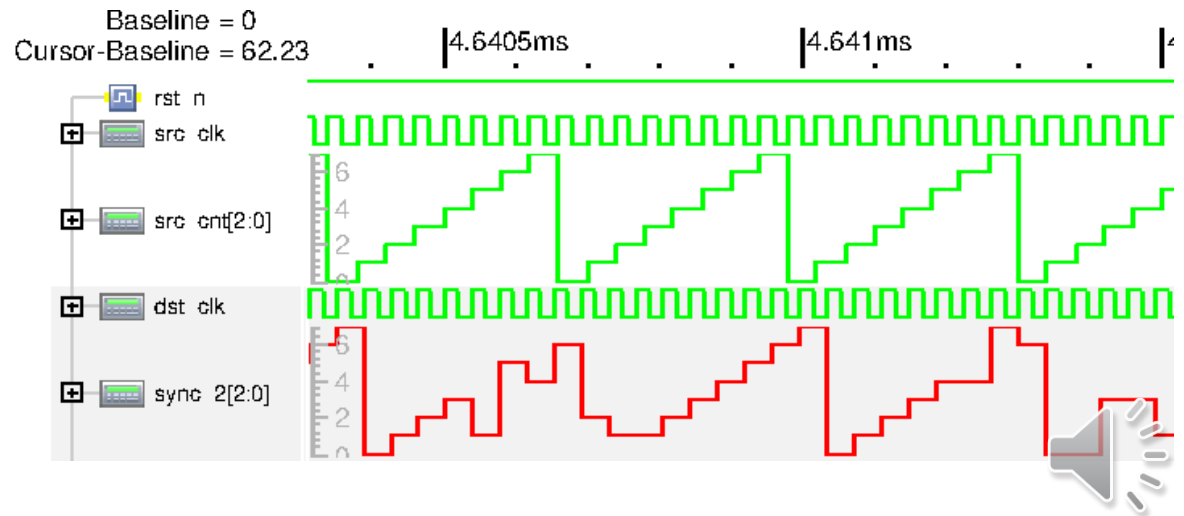
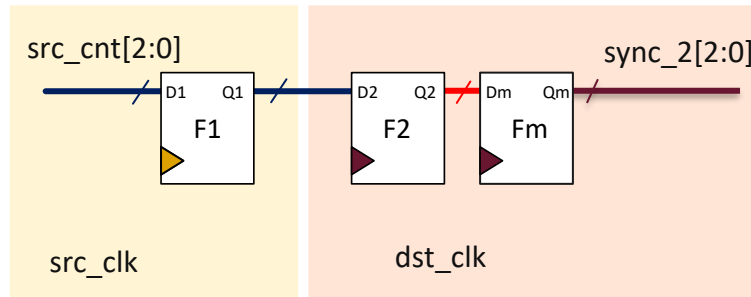
- **Why do we need synchronization**
 - Clock domain crossing, data from one clock is captured (sampled) in another clock domain
 - Clock domains are groups of clocks that are synchronous to one another
 - Metastability is observed during crossings if they are not synchronized



Multibit bus needs care

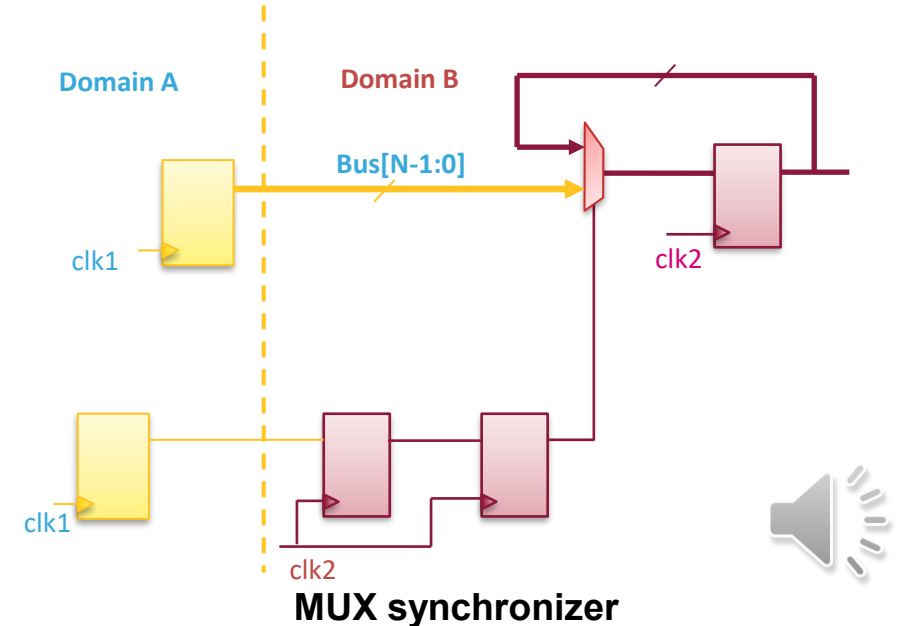
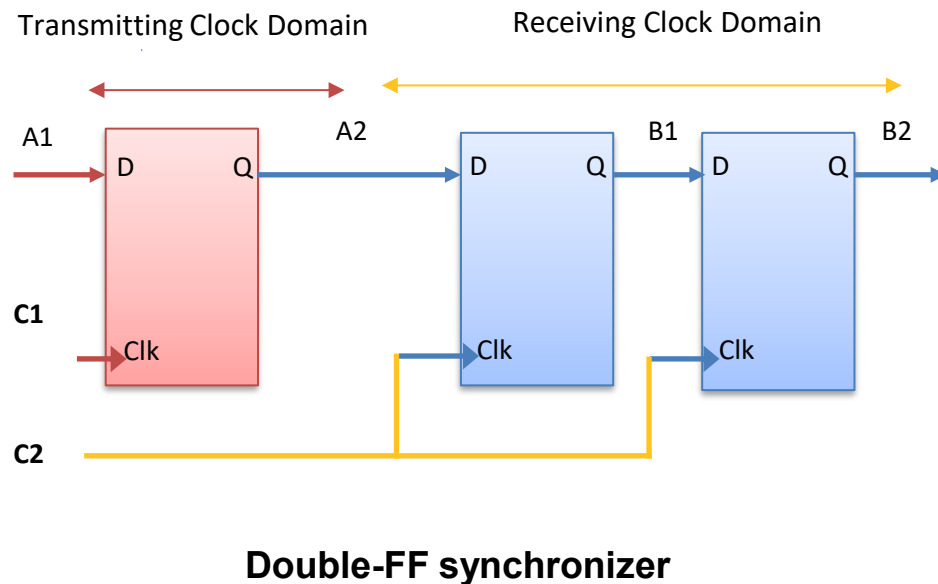
- **Multibit bus**

- With double FF (DFF), we can address metastability problems, but there might be data coherency problems
- If multiple bits change at the same cycle, the result of each bit is random by the DFF
- This synchronization works only if the data is gray coded (one bit change)



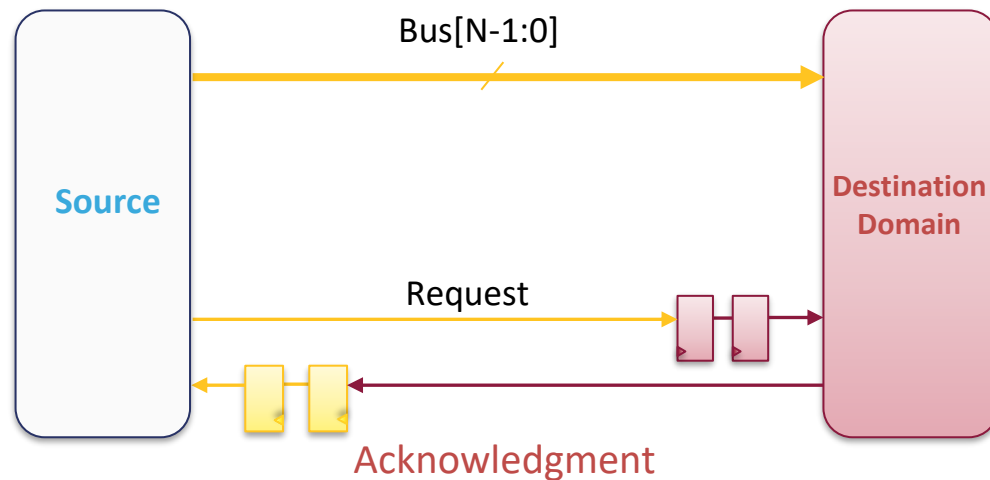
Different CDC schemes

- DFF is most commonly used synchronizer
- Optimized by design companies and included in libraries
- Mux synchronizer is another technique used for multi-bit buses

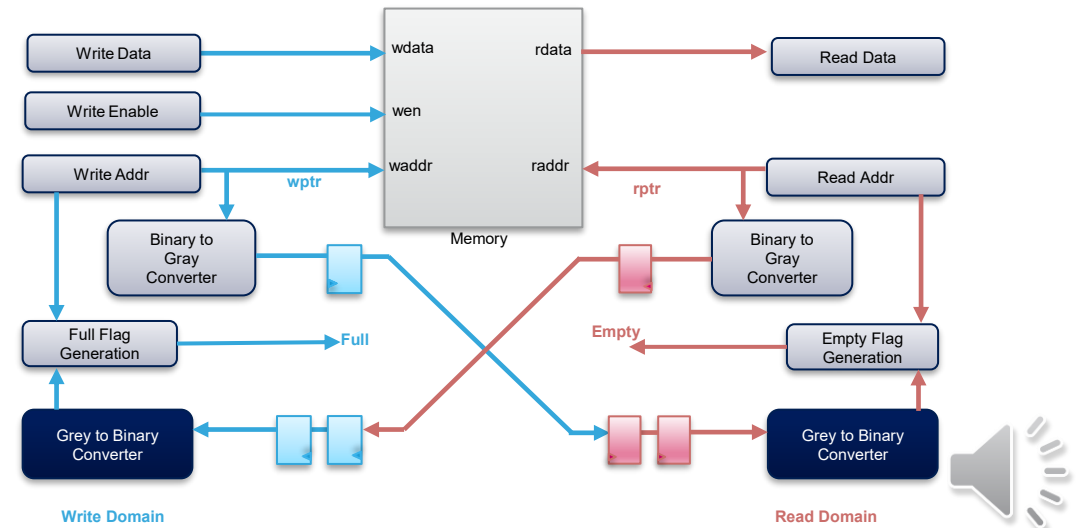


Different CDC schemes

- We have handshake synchronization where the TX and RX talk with req and ack for seamless data transfer
- FIFO synchronization, as you all know, is widely used, where the rd and wr pointers are synchronized across the domains



Handshake synchronizer



FIFO synchronizer

Tools for CDC analysis

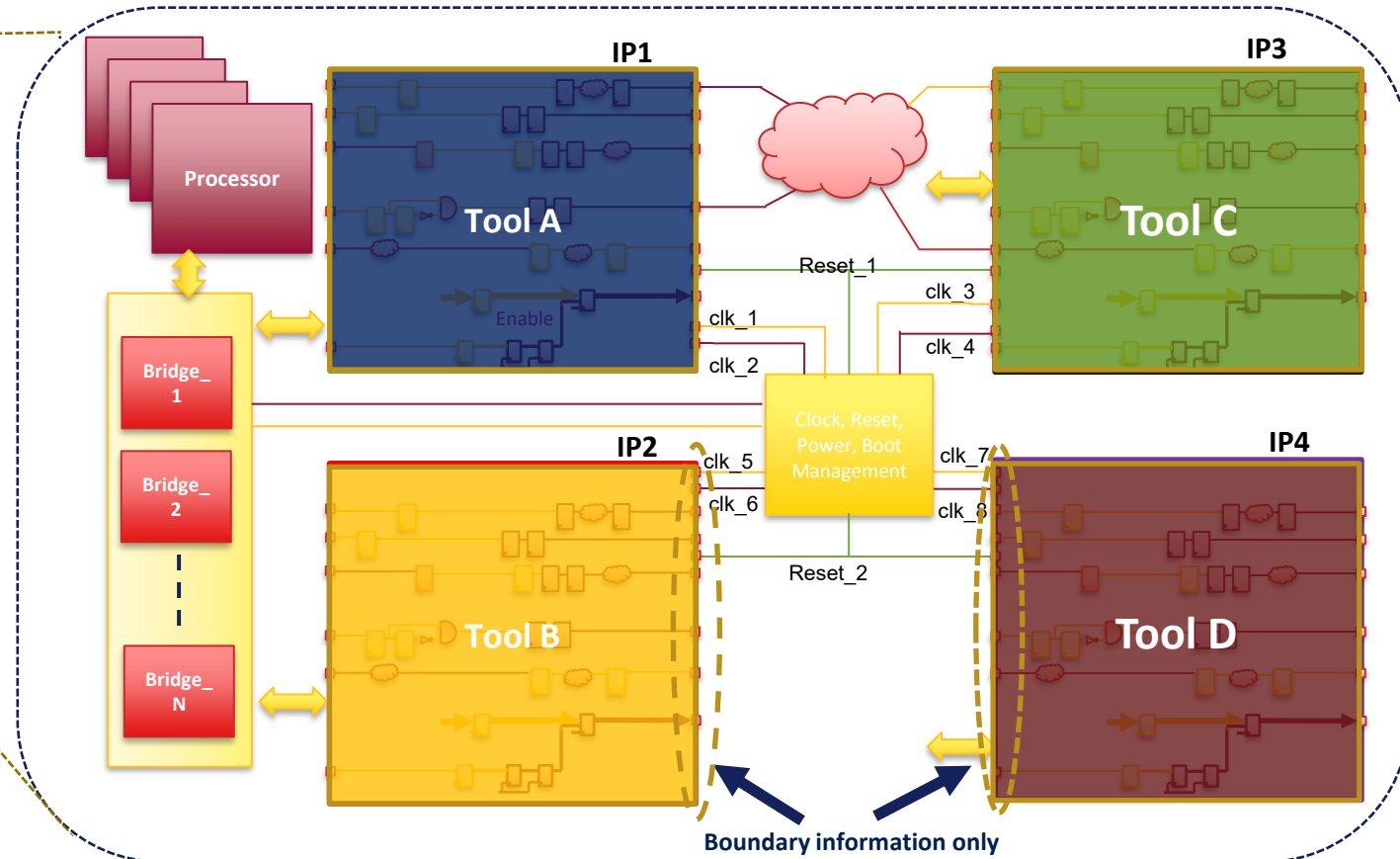
- Industry tools are used to perform static analysis on the CDC to find CDC bugs
 - CDC bugs: Design mistakes that allow metastability or inconsistent sampling to cause functional failures
- The tools examine every clock-domain crossing path and its synchronization structure to identify potential CDC issues.
- As the design complexity increases, the number of combinations grows exponentially, leading to significant run times
- The tools often require manual constraints to manage noise and false violations
 - Static, constant, etc.
- CDC analysis tools will:
 - Identify clock domains → find crossings → classify each crossings → check synchronization
- Common CDC analysis tools
 - Spyglass CDC, Questa CDC, Meridian CDC



Hierarchical flow in CDC analysis



- Bottom-up approach
- IP-to-IP paths need to be taken care from CDC
- CDC models are currently lacking standardization.
- CDC models from different tools are not compatible.



CDC and RDC standard



Standard for IP Abstraction for Clock and Reset Domain Crossing Integration Version 1.0

March 2026

Download Clock Domain Crossing

The "Standard for IP Abstraction for Clock and Reset Domain Crossing Integration" enables interoperable, tool-independent CDC/RDC collateral exchange and signoff from IP to SoC design flows.

CDC/RDC verification is essential to avoid metastability, glitches, and data reconvergence errors when signals traverse asynchronous clock or reset boundaries. Historically, analysis results and 'intent' generated by different tools could not be reused across different EDA tools, forcing re-runs and manual translation during integration. The standard defines a vendor-neutral approach to capture CDC/RDC intent to enable efficient CDC/RDC signoff for IP providers and SoC integrators, across the EDA vendor ecosystem.

The Standard for IP Abstraction for Clock and Reset Domain Crossing Integration was developed by the [CDC Working Group](#).

Download Standards

Current Release

Item	Download	Date Modified
Clock Domain Crossing	Standard for IP Abstraction for Clock and Reset Domain Crossing Integration 1.0	2026-03-02



<https://www.accellera.org/downloads/standards/clock-domain-crossing>

Agenda

- Introduction
- Timing basics
- Clocking
- CDC and Synchronization
- **RDC and resolving RDC**



Synchronous vs Asynchronous Reset

- Synchronous reset

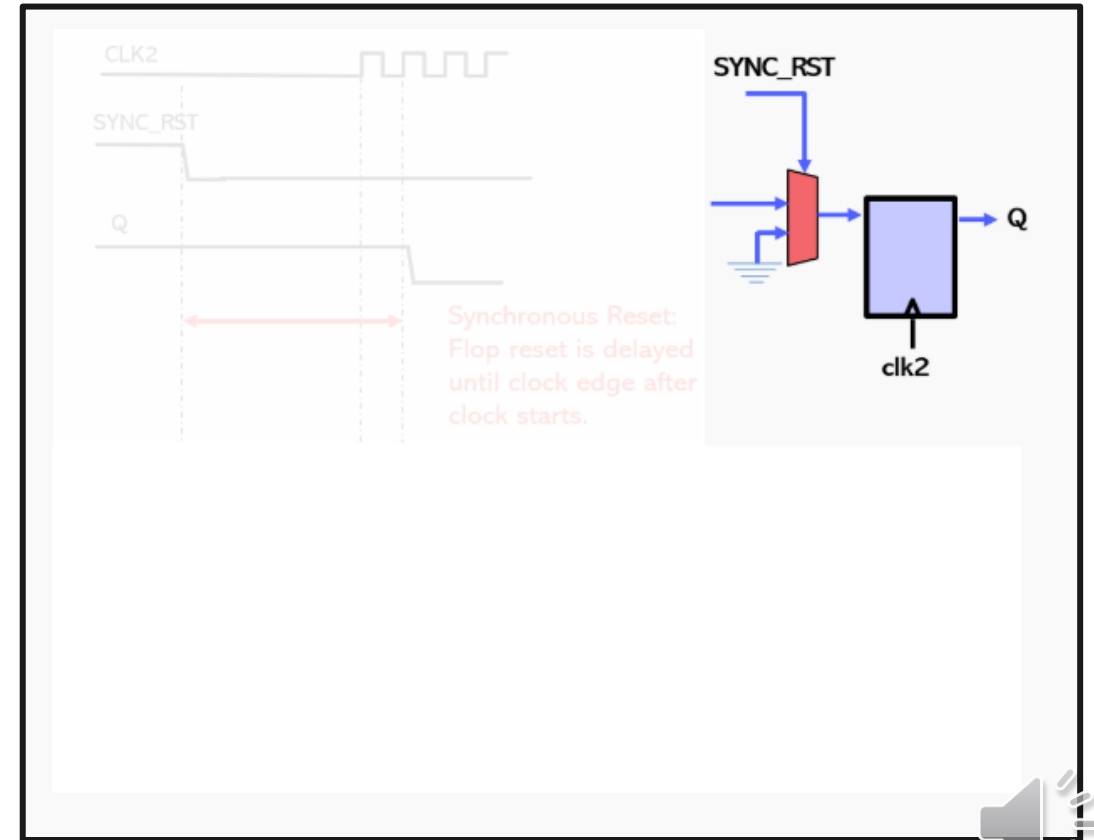
always **@(posedge clk)** begin

if (!rstn) q<='0;

else q<=d;

end

- Works only when the clock is toggling
- Need for additional gates outside the FF for blocking the data
- Need the clock to be running to drive the flop to its reset state, which will delay the reset until the next clock edge



Source: <https://besttechviews.com/reset-domain-crossing-asynchronous-resets-2/>

Synchronous vs Asynchronous Reset

- Asynchronous reset

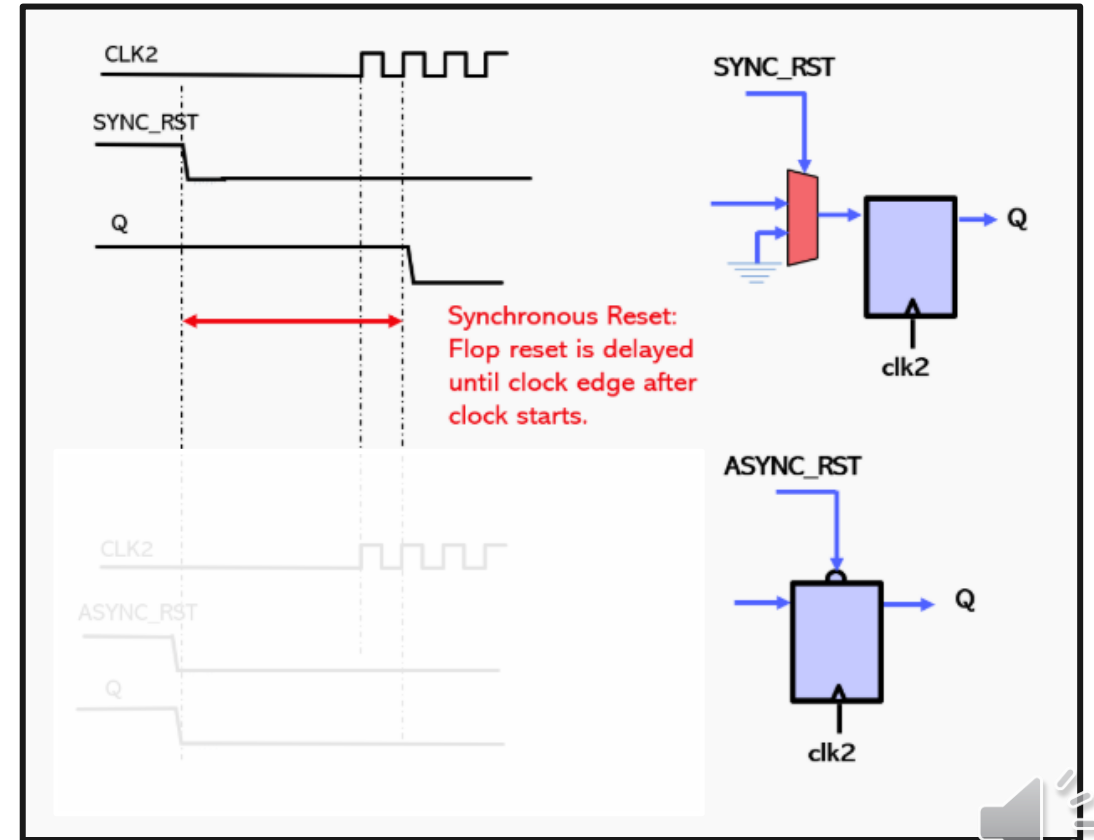
always **@(posedge clk or negedge rstn)** begin

if(!rstn) q<='0;

else q<=d;

end

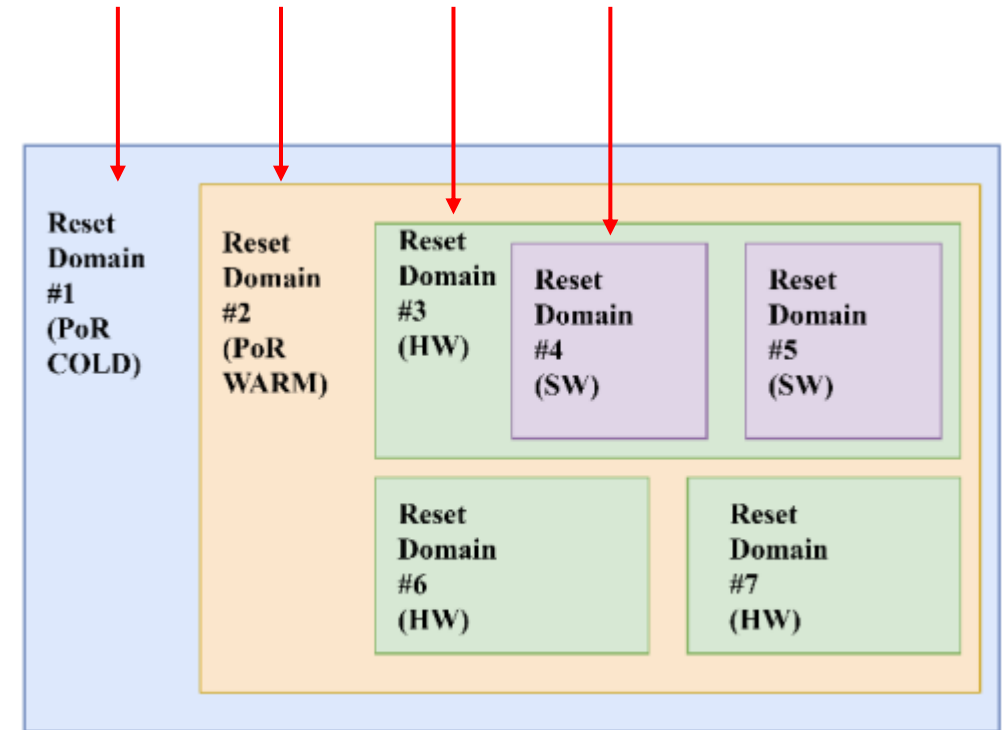
- No need for additional logic in data path
- Async resets are independent of clock toggling
- Consume less power and area compared to a sync reset
- Accelerates reset sequence, since it drives the FF to reset state immediately



Source: <https://besttechviews.com/reset-domain-crossing-asynchronous-resets-2/>

Types of reset in a design

- **Cold Power on reset**
 - Cold power-on-reset is done when the chip is completely powered off, and the power is then reapplied to initialize the system
- **Warm Power on reset**
 - Warm power-on-reset is performed when the chip is already operational, and a restart is initiated without completely taking away the power
- **Hardware HW Reset**
 - These resets are automatically triggered by built-in hardware mechanisms in response to certain events/conditions.
- **Software SW Reset**
 - Software-triggered resets have a more limited impact on the system-on-chip (SoC) compared to hardware resets.



Source: <https://iccircle.com/static/upload/img20250107182411.pdf>



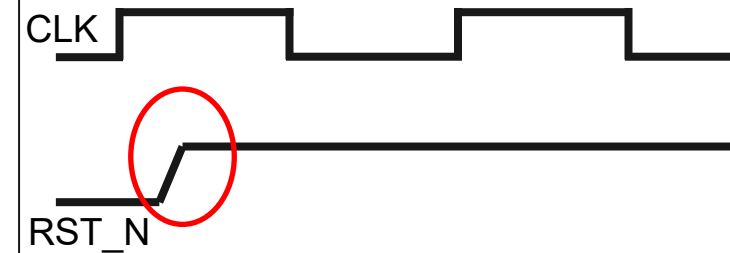
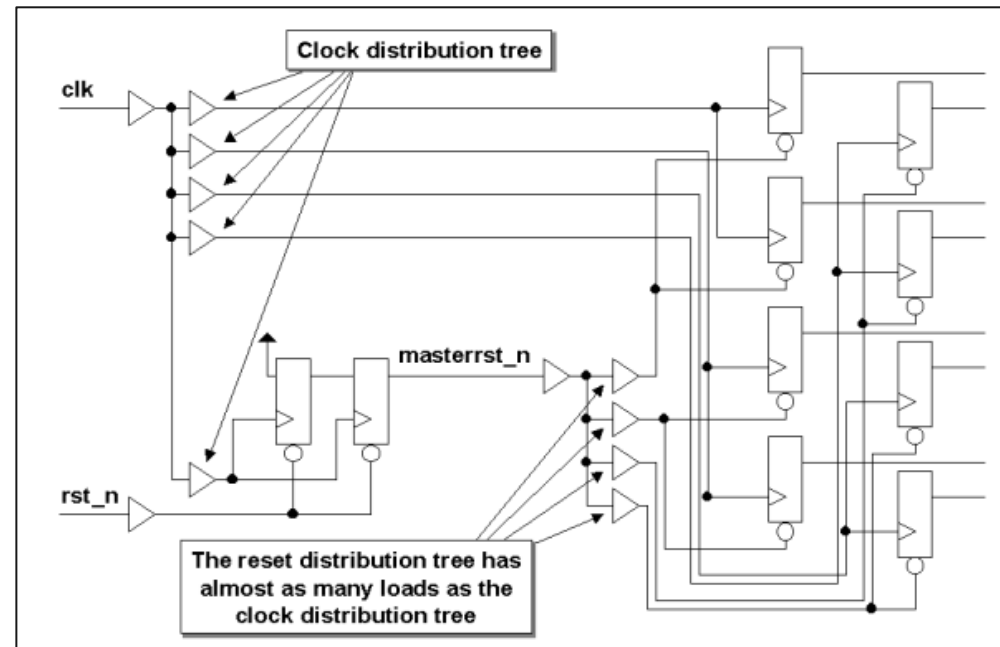
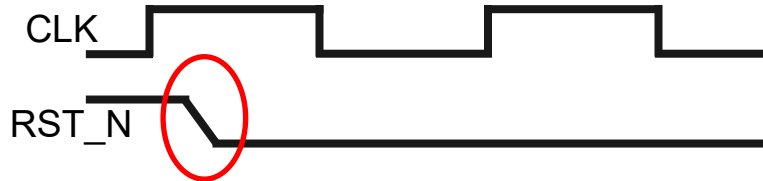
Reset assertion and deassertion

- **Reset assertion**

- Here we make the design or IP or block go into reset state by asserting the reset ($1 \rightarrow 0$) [active low reset]
- Can be asynchronously asserted

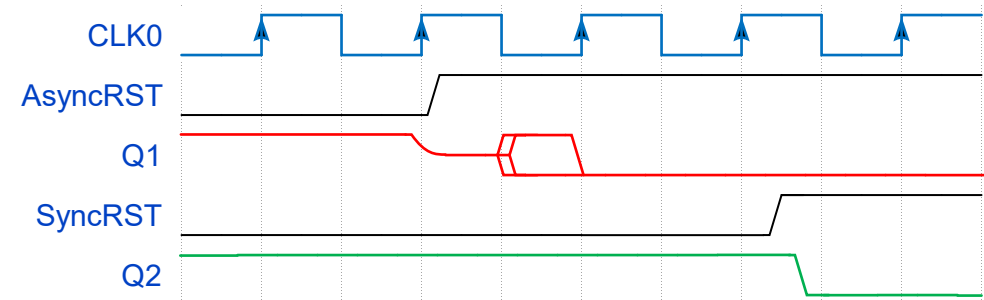
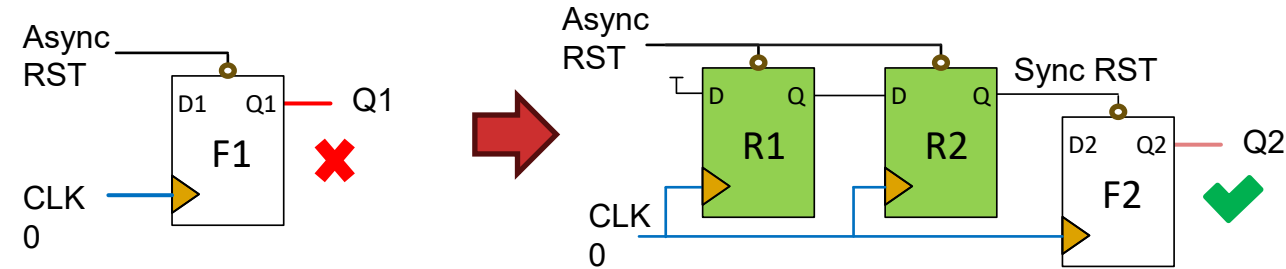
- **Reset de-assertion**

- Deassertion needs to have a synchronizer to avoid metastability and timing issues ($0 \rightarrow 1$) [active low reset]
- Synchronous deassertion

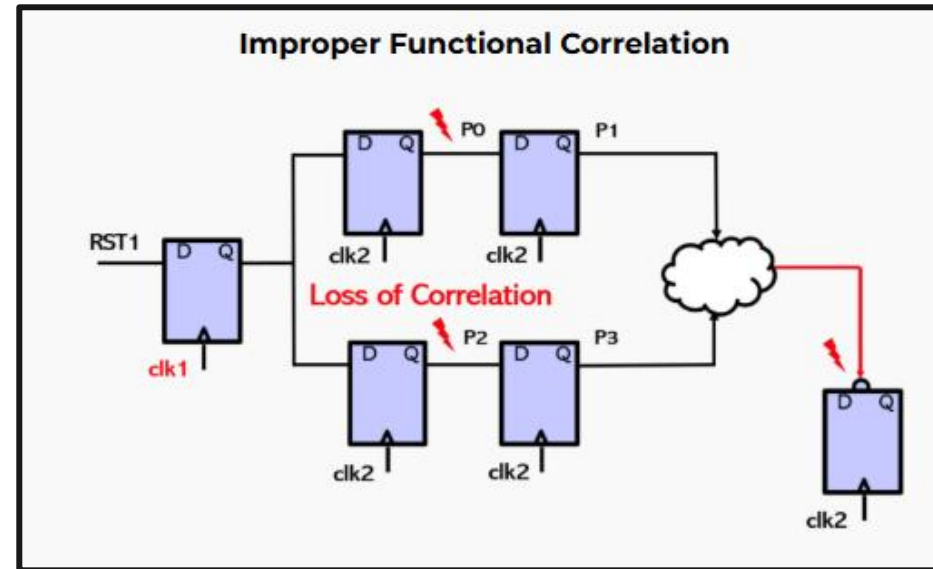
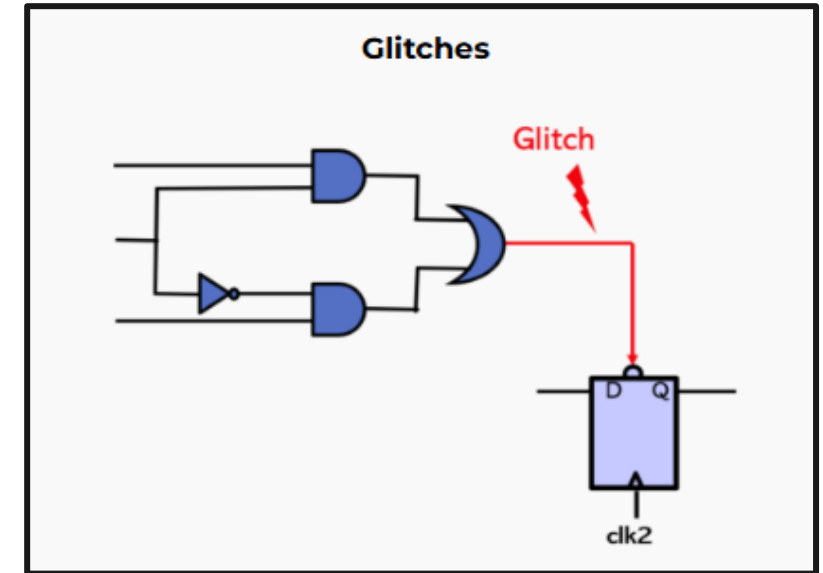
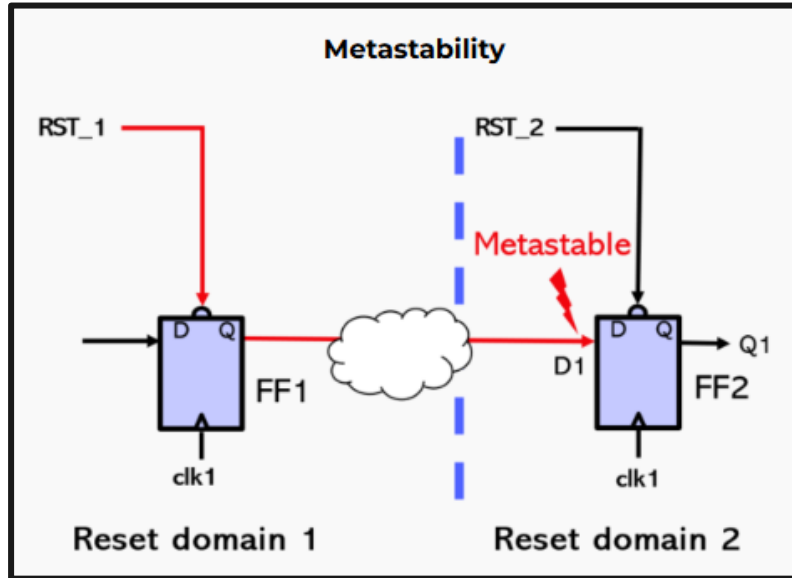


Reset deassertion issue

- **In addition to setup- and hold-, DFF models also have recovery-time**
 - Time between asynchronous Set/Reset release and clock when data and output are different
 - Violating recovery time is no different than violating setup/hold
- **Possible to synchronize asynchronous reset on release edge only**
 - Static analysis is sufficient to make this determination



Reset domain crossing (RDC)

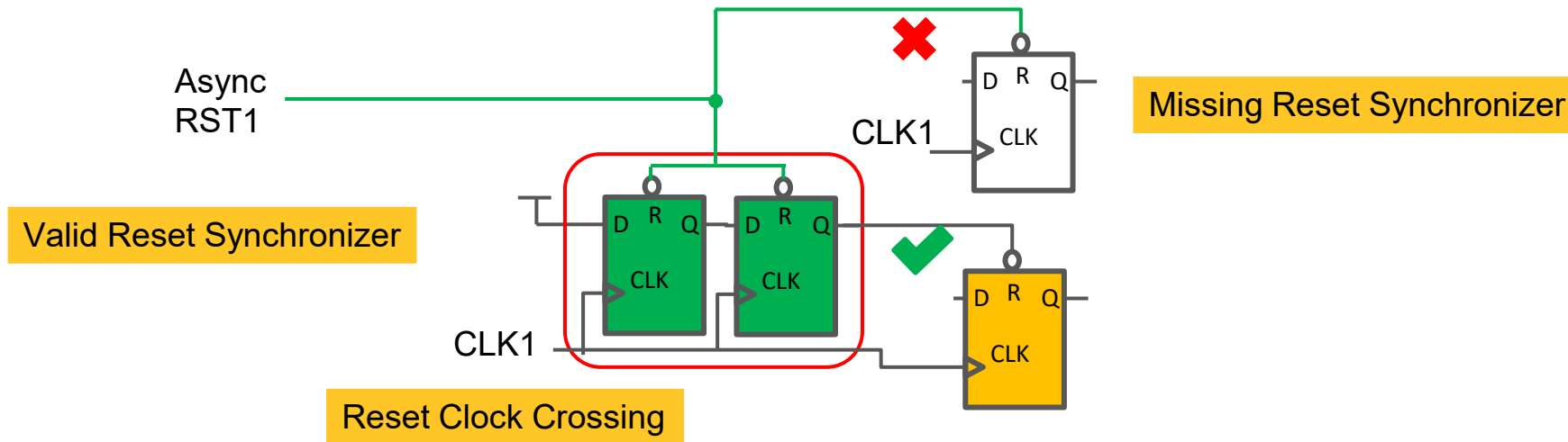


Reset domain crossing (RDC)

How are these resolved??



Reset Synchronizers

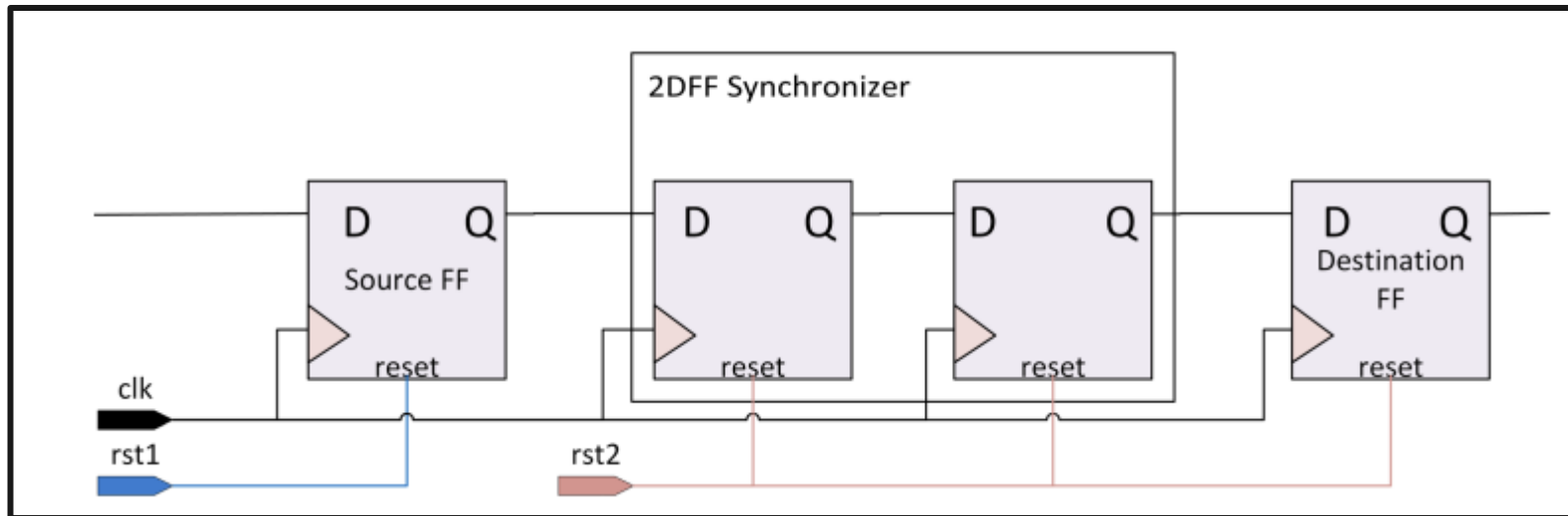


- **Reset signal crossing from one clock domain to another**
 - The asynchronous de-assertion of the reset signal at the destination flop can cause the signal to become metastable
 - Reset signals are required to be synchronized to destination domains for synchronous de-assertions



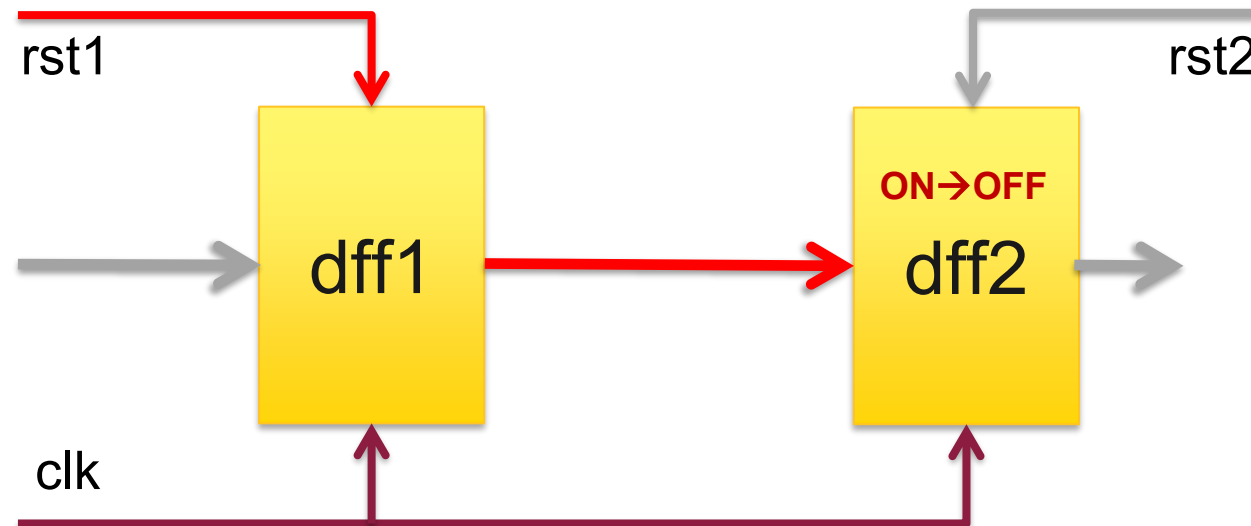
Resolving RDCs

- Resolving RDC issues
 - Adding synchronizer in the destination domain
 - Reset ordering of different resets in the design



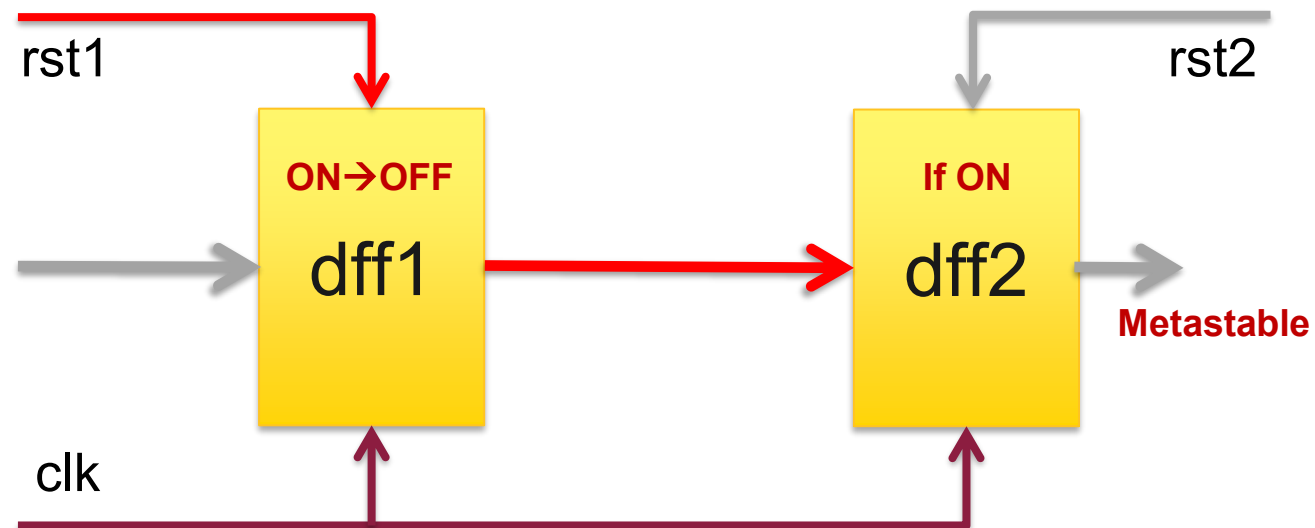
Reset ordering

- Make sure we have some reset conditions between the multiple resets we have in the design
- When rst2 asserts, we do not care if rst1 is asserted or deasserted



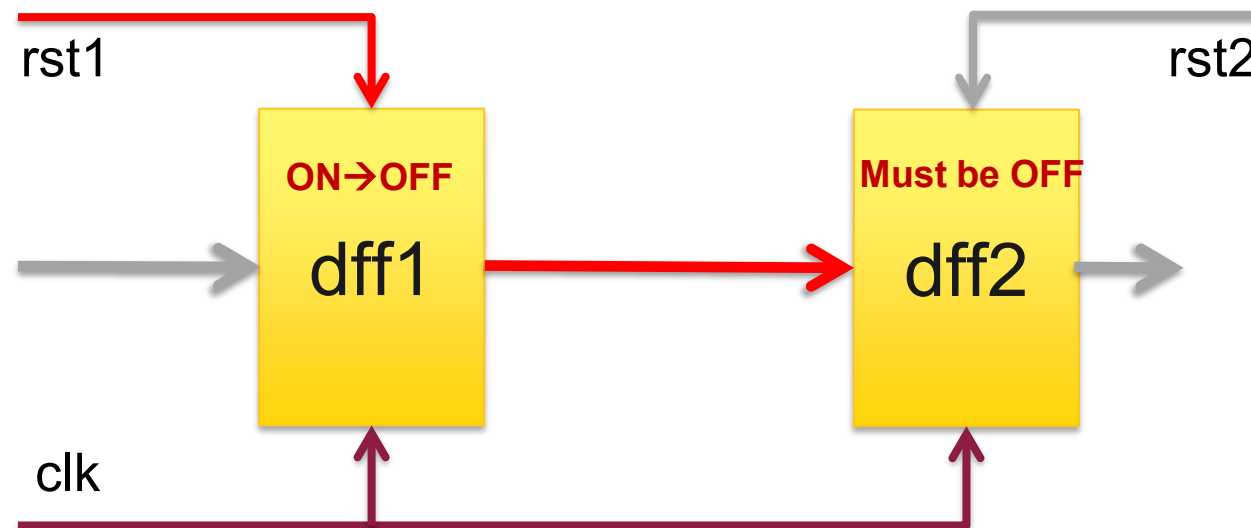
Reset ordering

- Make sure we have some reset conditions between the multiple resets we have in the design
- When rst2 asserts, we do not care if rst1 is asserted or deasserted
- But when rst1 asserts, we need to make sure rst2 is asserted



Reset ordering

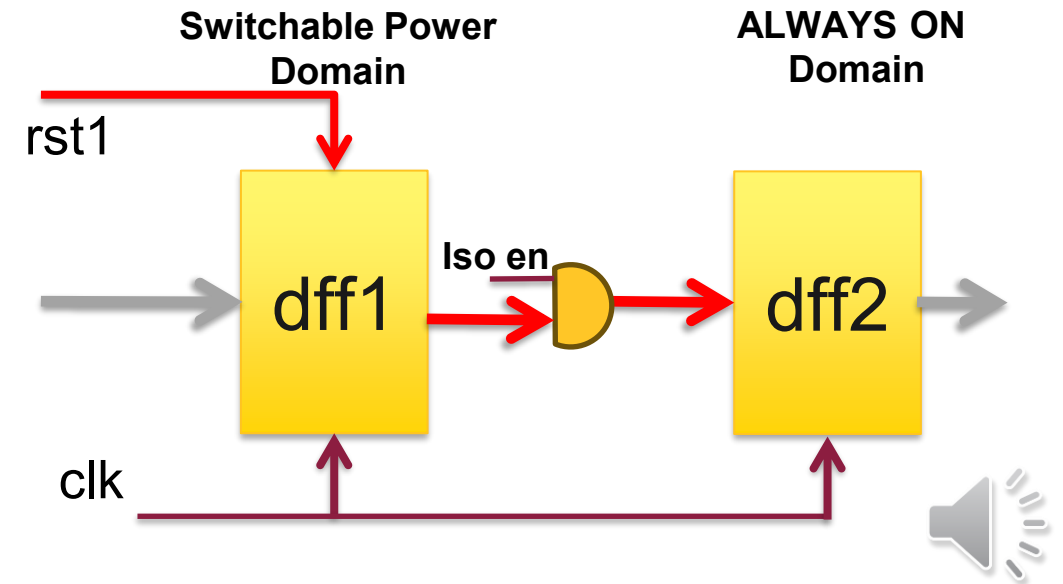
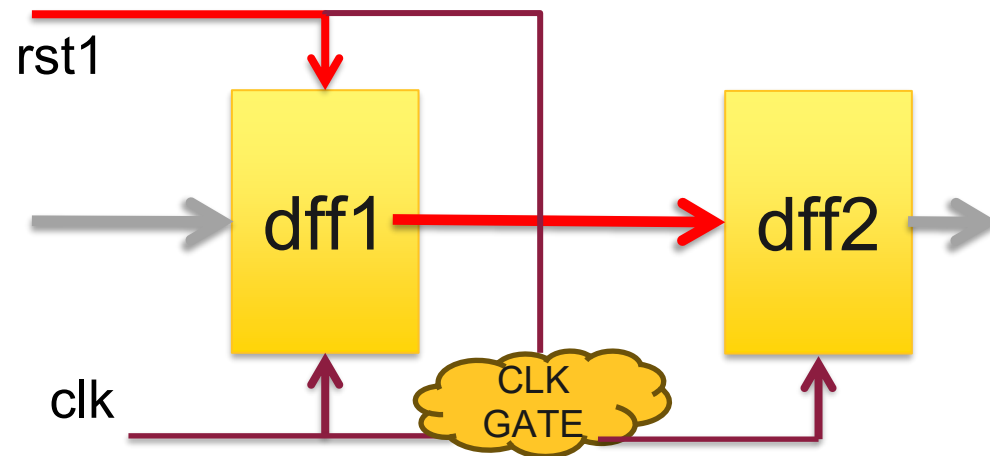
- Make sure we have some reset conditions between the multiple resets we have in the design
- When rst2 asserts, we do not care if rst1 is asserted or deasserted
- But when rst1 asserts, we need to make sure rst2 is asserted



Resolving RDCs

- **Resolving RDC issues**

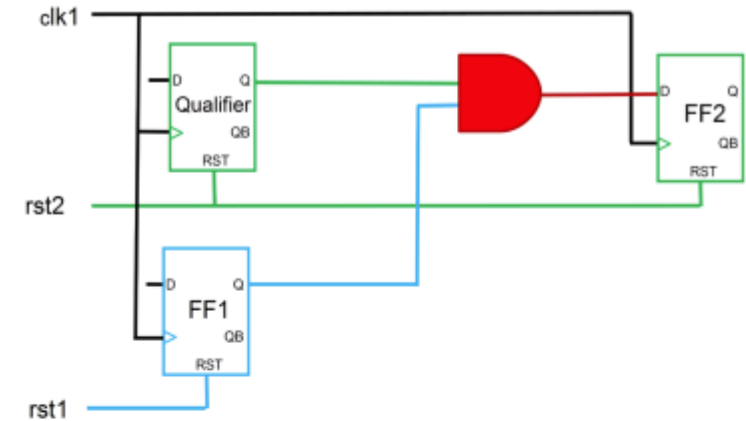
- Reset-less paths: Clock gate the downstream flops for some cycles when rst1 asserts to avoid glitch capture
- Multipower domain designs: reset ordering, isolation enable asserted



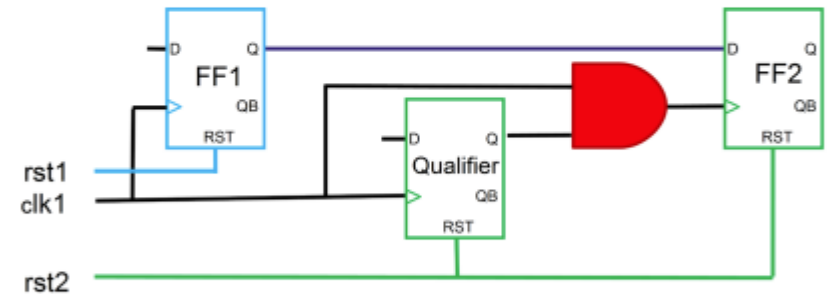
Resolving RDCs

- We have three types of qualifiers
 - Data qualifier
 - Clock qualifier
 - Enable qualifier
- Ensure the qualifier must arrive before the reset assertion edge
- Qualifier must be sampled in RX that is reset, and the clock of the receiving domain

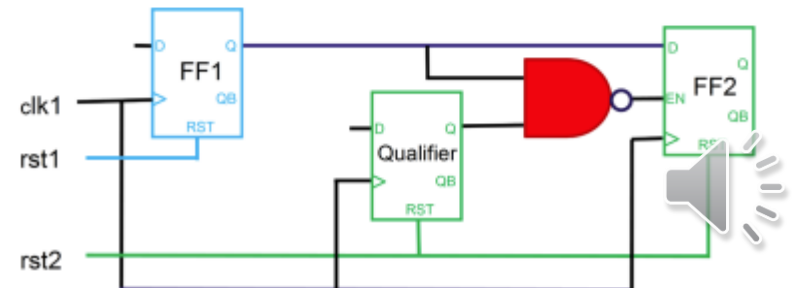
Data Qualifier



Clock Qualifier



Enable Qualifier

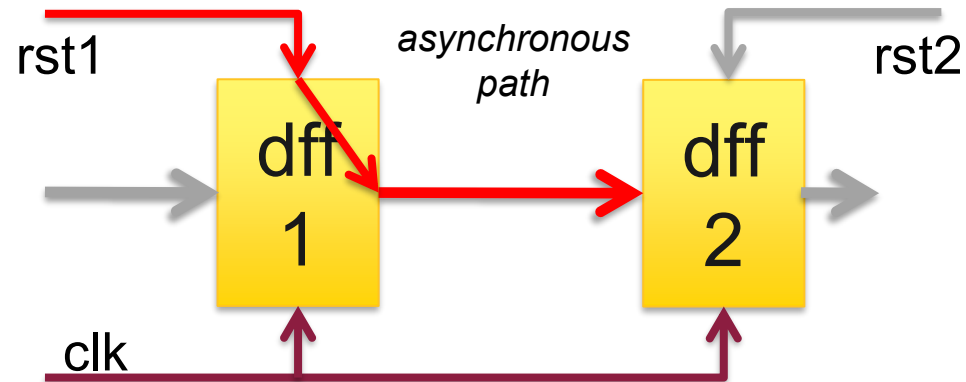


Thank you!



Reset domain crossing (RDC)

- **Asynchronous reset domains causes meta-stability**
 - Contain registers whose resets are asserted asynchronously
 - Originate in one asynchronous reset domain
 - Sampled by register(s) in a different reset domain
 - **Reset ordering of different resets in the design**



Backup-Timing basics

$$\boxed{T + T_{skew}} \geq T_{clk-Q} + T_{comb} + T_{slack} + T_{setup}$$

$$T_{clk-Q} + T_{comb} \geq T_{hold} + T_{skew}$$

